TroubleShooting MySQL Performance Issues

Debugging Performance Issues:

1. Slow query → analyse check logs -> login and check slow-query logs
2. Run Debug commands → analyze processlist, innodbstatus, rouge connection(kill them if needed)
    a. If not integrate with db creds
    b. If with Db creds

FAQS:

Problem/Issue	Potential Causes of the Issue	Solutions/Suggestions	Brief Description
Deleting data from mysql tables, disk space is still not freed up	InnoDB marks deleted data as free space but doesn't release it to the OS	Use `ALTER TABLE table_name ENGINE=InnoDB;` to optimize the table and reclaim space. Check for Datafree: `SELECT CONCAT(table_schema, '.', TABLE_NAME) AS 'Table', CONCAT(ROUND((data_length + index_length) / (1024 * 1024), 3), 'G') AS 'Table Size', CONCAT(ROUND((data_free / 1024 / 1024/1024),2), 'GB') AS 'Data Free', CONCAT('(', IF(data_free< (data_length + index_length), CONCAT(round(data_free/(data_length+index_length)*100, 2), '%'), '100%'), ')') AS 'Data Free Pct' FROM information_schema.TABLES WHERE table_schema not in ('mysql','performance_schema','sys','test','information_schema') ORDER BY ROUND(data_free / 1024 / 1024/1024, 2) DESC LIMIT 10;`	InnoDB doesn't immediately free disk space after deletion; optimization is needed to reclaim it
Increase in QPS despite a constant number of	QPS can increase due to optimized queries, caching, or	Analyze the 'Top Command Counters' graph in the MySQL	Increased QPS isn't necessarily bad; it indicates

connections	increased workload from the application side	Detailed Dashboard via PMM to identify the cause. Ensure no performance bottlenecks.	more processed queries. Investigate only if it causes performance issues.
Tuning query and transaction timeouts in MySQL cluster	Default timeout settings may not be optimal for workload demands	Max_execution_time, Kill_idle_transaction,innodb_lock_wait_timeout, innodb_rollback_on_timeout, wait_timeout	Configuring timeout variables properly ensures efficient query execution and transaction management
Determining the maximum QPS an instance can achieve without bottlenecks	QPS depends on factors like query type, workload, and system resources	Perform benchmarking with your workload to identify the max QPS for your instance	QPS varies by workload and hardware; benchmarking is needed to determine actual limits
Compress/decompress a table in MySQL	Tables may consume excessive storage space without compression	`For compression: ROW_FORMAT=COMPRESSED OR KEY_BLOCK_SIZE=8 For decompression: ROW_FORMAT=DYNAMIC OR KEY_BLOCK_SIZE=16`	MySQL supports InnoDB table compression to optimize storage.
Choosing a partition management option for MySQL tables	Depends on use case: incremental (e.g., `unix_timestamp()`) vs. circular (e.g., Monthly/Weekly)	For circular partitions, truncate old partitions timely. For incremental, actively manage partition creation and deletion. Implement partition pruning to optimize performance. Keep a last partition (`PMax`) to prevent write downtime	Proper partition management ensures efficient data handling and avoids performance degradation
"Temporary file write failure" error during ALTER operation	Table size exceeds allocated temporary space (`/var/tmp/mysql`) during online-DDL ALTER	Use the `pt-osc` tool to avoid tmp space exhaustion. Refer to MySQL ALTER table documentation for alternative approaches	Large table alterations can fail due to tmp space limits; pt-osc helps prevent this issue

"Too many connections" error in MySQL	The `max_connections` limit is reached, preventing new connections	Increase `max_connections` to allow more concurrent connections. Check and optimize connection pooling in the application	MySQL limits connections via `max_connections`. Adjusting this setting prevents connection failures
How to reorganize a partition in MySQL	Large data movement occurs during partition reorganization	Purge old data and optimize the table before reorganizing. Use online DDL or pt-osc command for REORGANIZE for large tables	Reorganizing partitions redistributes data for better efficiency. prevents performance issues
Data deletion guidelines in MySQL	Large-scale deletions can impact performance and disk space	Use tools like `pt-archiver`, Flipkart Scavenger, or controlled delete statements (<10K records per batch). Delete using primary keys and avoid subqueries. Optimize tables after deletion to reclaim space	Proper deletion practices prevent performance issues and free up storage efficiently
Error while adding a node (e.g., optimizer) to a cluster	`innodb_buffer_pool_chunk_size` too high	Increase `innodb_buffer_pool_chunk_size` (e.g., to 1GB)	Adjusting buffer pool chunk size resolves the error
Online DDL error: `innodb_online_alter_log_max_size` exceeded	Insufficient `innodb_online_alter_log_max_size`	Increase `innodb_online_alter_log_max_size` (e.g., to 2GB)	Expanding the log size prevents alter failure
Request to reduce disk space in LVM partitions	LVM partitions cannot be shrunk, only extended	Create a new cluster with correct sizes and migrate via A-A flow	LVM resizing requires reconfiguration
Packet for query is too large error	`max_allowed_packet` is too small	Increase `max_allowed_packet` (up to 1GB)	Adjusting packet size resolves data load failures

MySQL Error 1264: Out of range value for a column	Column datatype too small for inserted values	Use `UNSIGNED` or change datatype (e.g., `INT → BIGINT UNSIGNED`)	Ensures data type compatibility
No space left on device error	Full `/var/tmp/mysql` partition due to temp files	Increase partition size, optimize queries, adjust `tmp_table_size` and `max_heap_table_size`	Avoids MySQL temp file failures
Connection break error (timeout reading packets)	`wait_timeout` too low or inefficient connection pooling	Set `wait_timeout` correctly, reuse connection pool, increase `innodb_lock_wait_timeout`	Prevents unnecessary connection drops
Disk usage divergence (10%) in MySQL 5.7	Normal behavior due to MySQL version	Use purging, optimization, table compression	Disk divergence is expected in MySQL 5.7
max_connection is too high for available memory error when adding a node	Insufficient memory for max_connections setting	Adjust memory- related variables	Max connections depend on available memory
Migrating a single DB between clusters	System doesn't support selective DB migration	Use `mysqldump` for manual backup and restore	Only full cluster migrations are supported

How to track and a long running transaction in MySQL

Alert name: LongRunningTransaction
Details by using "**CALL ShowEngineInnoDBStatus();**" and "**CALL GetLongRunningTrx(n);**" stored procedures.

Stored Procedure	Description
CALL GetAllLatentQueries(10)	To list all queries taking more than 10 seconds
CALL GetHungTranasctions()	To list all hung transactions, Transaction is still running but corresponding mysql thread is in sleep state
CALL ShowEngineInnoDBStatus()	To list innodb status
CALL GetLongRunningTrx(300)	To get all transactions running for more than 300 seconds
CALL ShowProcessList()	To list all process running in the instance, Queries + Sleep Connections ...
CALL KillConnection(<ThreadId>)	To kill any mysql connection
CALL KillQuery(<ThreadId>)	To kill any mysql query, connection is not closed in the case

Troubleshooting Replication Lag

Causes of Replication Lag
MySQL's default async replication setup
High CPU & IOPS utilization
Hung transactions & long-running queries
Slow queries (>1 sec execution time)
Sudden spikes in active threads due to application issues

Steps to Monitor and Debug
Check replication lag in the Cosmos dashboard.
Analyze resource utilization (CPU & IOPS) in Cosmos graphs.
Identify and kill long-running or idle transactions.

Enable slow_query_log and optimize slow queries.

Tuning Parameters to Reduce Replication Lag
Ensure durability flags (sync_binlog, innodb_flush_log_at_trx_commit) are disabled in replicas.
Set kill_idle_transaction to automatically terminate idle transactions.
Set max_execution_time (e.g., 60000µs) to kill long-running SELECT queries.
Use binlog_format=ROW for better parallel replication (requires downtime).
Increase innodb_log_file_size (min 2GB) for performance improvement.
Tune binlog_group_commit_sync_delay (5-10ms) on the master to balance performance.
Control active threads from the application side & set innodb_thread_concurrency = 2 × VM CPU cores.
Adjust slave_parallel_workers (default = 8) for VMs with >12 cores (requires on-call support).

Mysql Best Practices and debugging

Pt-online-schema-change (pt-osc)
Percona tool that creates a copy of the table, applies changes, and swaps it with the original.

Pro	Cons
No downtime Various options to control process Estimated completion time Doesn't use temp partition space	Requires extra space in `/var/lib/mysql` Works with triggers, which can cause performance issues if not monitored Not recommended for tables with foreign keys

Online Alter
MySQL 5.7 supports online ALTER with the INPLACE algorithm. Uses both `/var/lib/mysql` and `/var/tmp/mysql` partitions.

Pro	Cons
No external tool needed No downtime Can be executed directly	No control over resource usage Can cause replication lag Uses both data and temp partition space No estimated completion time

Alter using Optimizer Node

System allows adding an optimizer node to perform alterations separately before rebuilding HSB and switching master.

Pro	Cons
No impact on production environment Can alter tables in replica separately	Not all ALTER operations are supported Requires stopping optimizer node replication Needs rebuilding HSB and master switch (downtime required) If replication breaks, the process must be restarted

Direct Alter in Master

Certain ALTER operations (e.g., adding a column, changing int to bigint) must be done in the master itself to avoid replication issues.

Pro	Cons
Required for certain schema changes	Impacts production directly May cause replication issues

Truncate Data

Steps:

1. Create a user with `DELETE` & `DROP` permissions.

2. If using **DBCreds**, create the user from the DBCreds portal.
3. Use `TRUNCATE TABLE table_name;` to remove all rows.

Considerations:

Performance Impact: In high-traffic systems, renaming and dropping the old table is recommended to avoid locks. Backup Recommended: Truncated/dropped data is permanently lost. Auto-increment Reset: Be cautious as it resets `AUTO_INCREMENT`, which may impact services.